# KCF Object Tracking



## 1. Course Content

- Run example programs, use the mouse to frame the object to be tracked in the screen, press the `spacebar`, the car enters tracking mode. The car will maintain a distance of 0.4 meters from the tracked object, and always ensure the tracked object stays in the center of the screen.

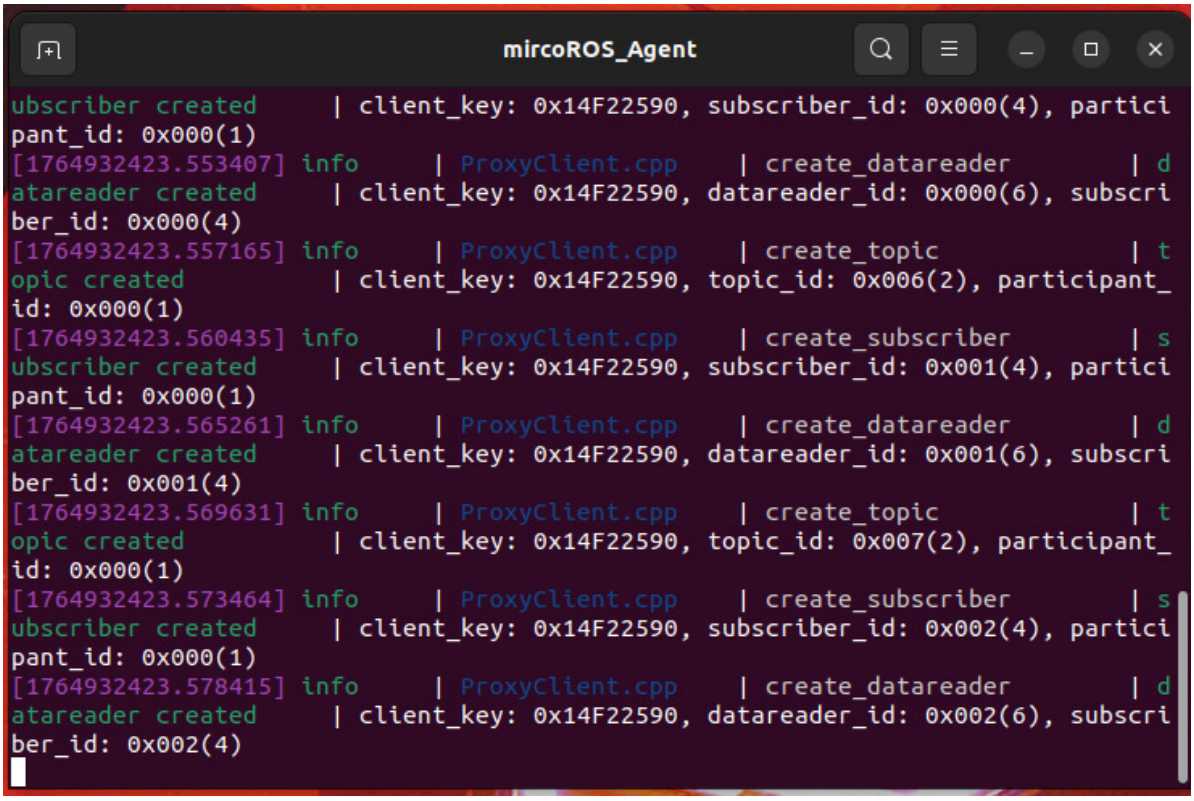
## 2. Start Agent

**Note: If already started, no need to restart**

Enter command in the car terminal:

```
start_agent
```

The terminal prints the following information, indicating successful connection




## 3. Run Case

**For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,**

```
bringup_m3
```

**Open a terminal inside the container,**

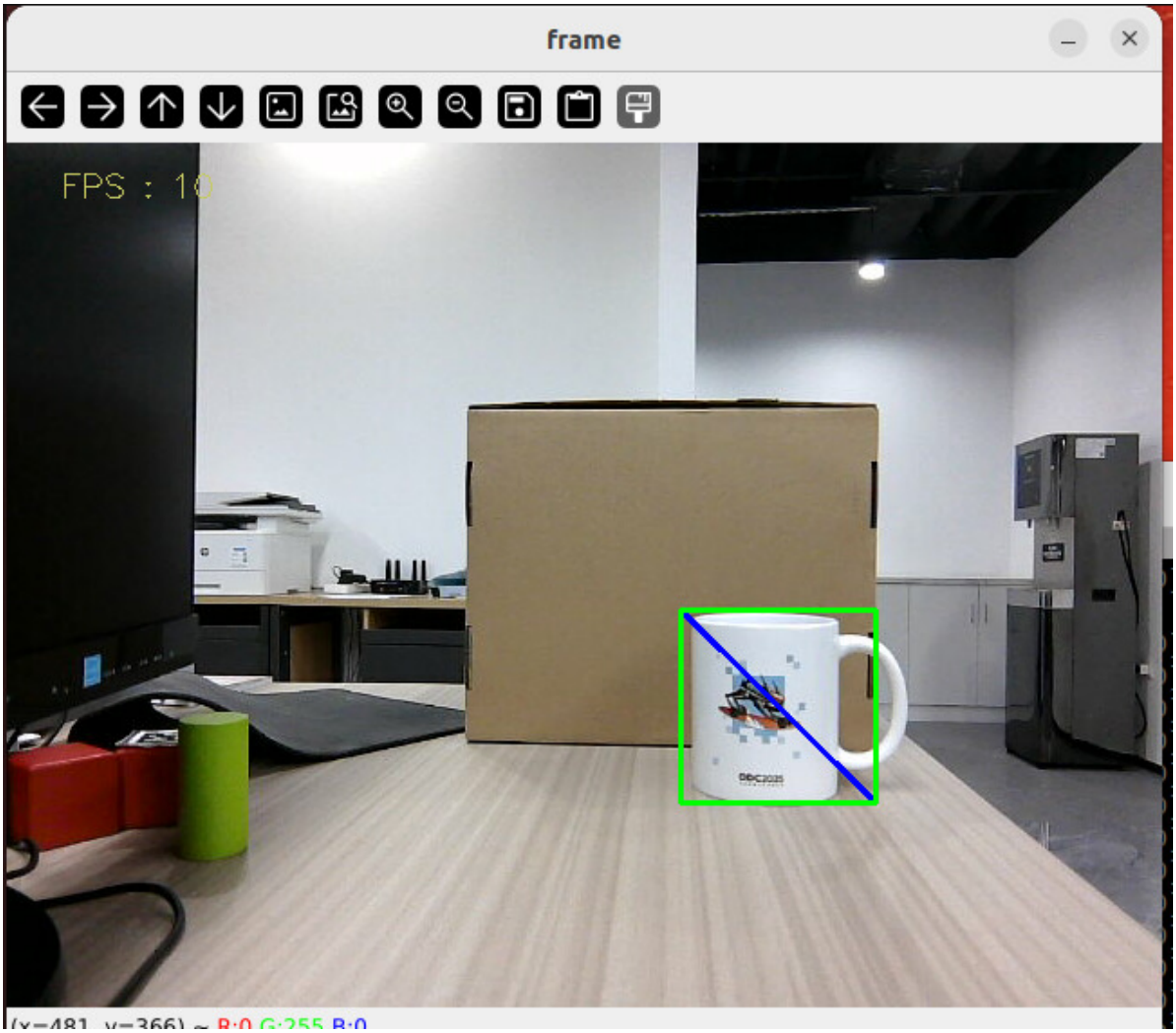
```
exec_m3
```

- Enter command in terminal:

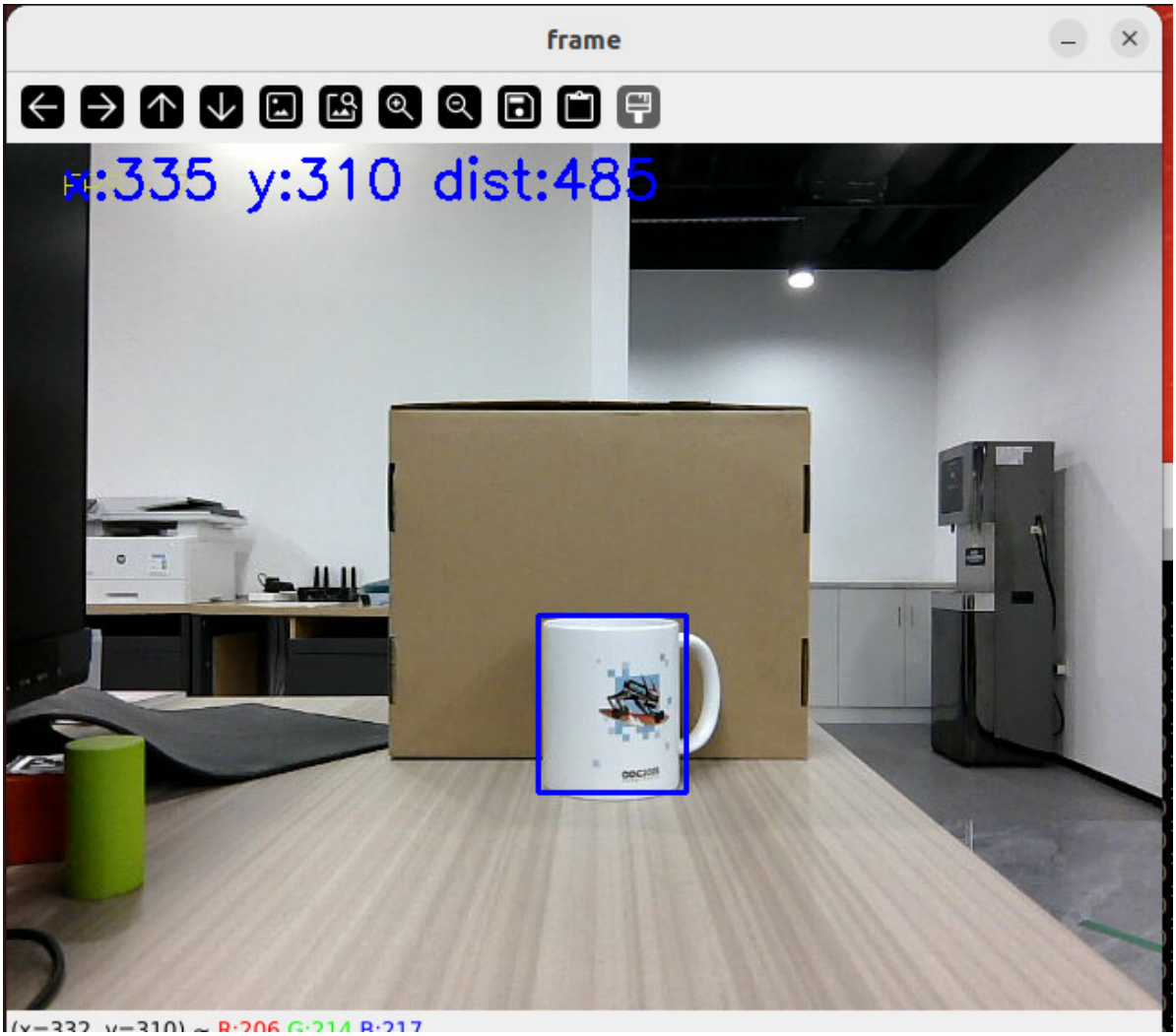
```
ros2 launch m3_bringup depthcam_demo.launch.py demo:=kcf_Tracker
```

- Use the mouse to frame the object to be tracked in the screen

> [!TIP]
>
> If it's easy to lose tracking, it's recommended to frame a larger and more obvious object

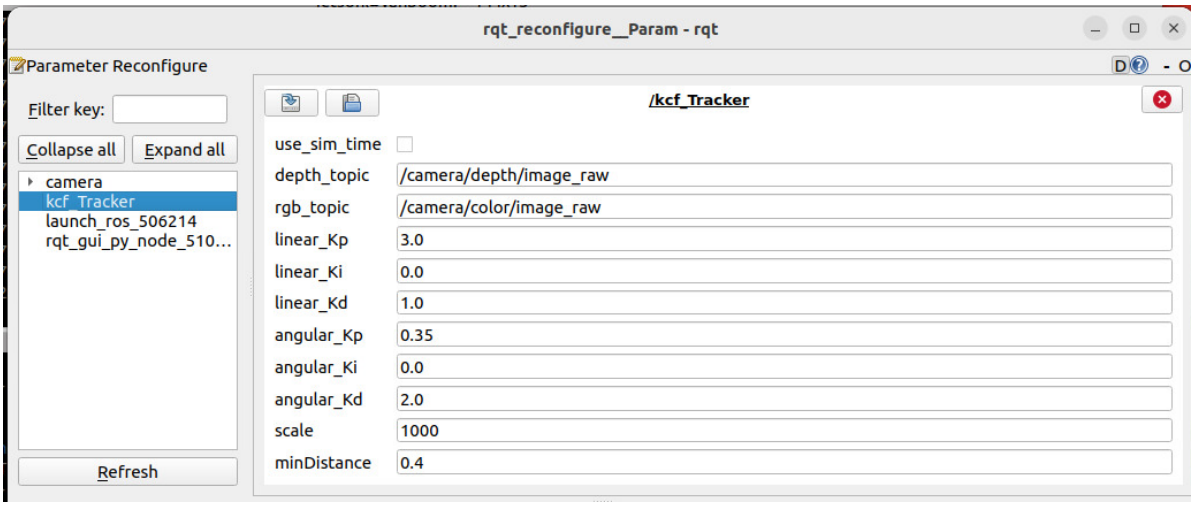



- Press the `spacebar` to enter following mode, slowly move the object, the car will follow the movement and maintain a distance of 1m from the object.




## 3.1 Dynamic Parameter Adjustment

```
ros2 run rqt_reconfigure rqt_reconfigure
```




- After modifying parameters, click on the blank area of the GUI to write parameter values. Note that it will only take effect for the current startup. For permanent effect, you need to modify the parameters in the source code.

**Parameter analysis:**

【linear_Kp】, 【linear_Ki】, 【linear_Kd】: Linear speed PID control during car following process.

【angular_Kp】, 【angular_Ki】, 【angular_Kd】: Angular speed PID control during car following process.

【minDistance】: Following distance, always maintain this distance.

# 4. Source Code Analysis

`KCF_Tracker.py` source code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/depthcamera/KCF_Tracker.py
```

- This program mainly has the following functions:
- Initialize KCF tracker
- Subscribe to depth topics to get images
- Select tracking target through mouse interaction
- Calculate target center coordinates and publish
- Use PID algorithm to calculate servo angle, forward speed and publish control commands

Some core code is as follows,

```
#Tracking initialization
self.tracker_type = 'KCF'

#Distance calculation
points = [
        (int(self.cy), int(self.cx)),          # Center point
        (int(self.cy +1), int(self.cx +1)),  # Lower right offset point
        (int(self.cy - 1), int(self.cx - 1))   # Upper left offset point
    ]
valid_depths = []
for y, x in points:
    depth_val = depth_image_info[y][x]
    if depth_val != 0:
        valid_depths.append(depth_val)
if valid_depths:
    dist = int(sum(valid_depths) / len(valid_depths))
else:
    dist = 0
#Control execution
if dist > 0.2:  # Effective distance threshold (0.2 meters)
    self.execute(self.Center_x, dist)  # Execution control
# Calculate the movement speed and turning angle using the PID algorithm based on the x and y values
def execute(self, rgb_img, action):
    #PID calculation deviation
    linear_x = self.linear_pid.compute(dist, self.minDist)
    angular_z = self.angular_pid.compute(point_x,320)
    # The car is in the parking area and the speed is 0
    if abs(dist - self.minDist) < 30: linear_x = 0
    if abs(point_x - 320.0) < 30: angular_z = 0
    twist = Twist()
    ...
    # Publish the calculated speed information
    self.pub_cmdVel.publish(twist)
```